

DATAImmo

Analyse du marché immobilier français

Conception et alimentation d'une base de données relationnelle à partir des Demandes de Valeurs Foncières — 1er semestre 2020

Projet OpenClassrooms · Parcours Data Analyst

Laplace Immo

1 Contexte du projet

3
Fichiers source

4
Tables créées

~180K
Transactions S1 2020

12
Requêtes d'analyse

🎯 Objectif
Concevoir une base de données relationnelle normalisée (3NF)
Alimenter la base à partir des fichiers DVF open data DGFiP
Permettre des analyses du marché immobilier par région, département et commune
Répondre à 12 requêtes d'analyse métier

🛠️ Outils utilisés
Power Query Nettoyage et transformation des données
SQLite Moteur de base de données
DB Browser Interface de gestion SQLite
SQL Power Architect Modélisation du schéma

🔄 Reproductibilité
📂 Sources de données
Valeurs-foncières (DVF) — DGFiP · Transactions immobilières S1 2020
Donnees-communes — INSEE · Population légale des communes
fr-esr-referentiel-geographique — MEN · Référentiel régions/communes

📐 Architecture
Schéma normalisé **3NF** : élimination de toutes les dépendances transitives
4 tables : REGION · COMMUNE · BIEN · VENTE
3 clés étrangères · Intégrité référentielle garantie
Gain volumétrique : **~90%** vs table unique

2 Stratégie de sauvegarde et conformité RGPD

🔒 **Donnée sensible identifiée et supprimée : "Nom de l'acquéreur"— suppression dès l'étape Power Query, avant tout traitement**

⚖ **Conformité RGPD**
Nom de l'acquéreur supprimé (donnée personnelle directement identifiante)
Suppression réalisée dès Power Query (étape 2 du nettoyage)
Aucune donnée nominative conservée dans la base finale
Les transactions restent analysables sans identifier les personnes

💾 **Stratégie de sauvegarde**
Fichiers sources conservés en version originale (non modifiés)
Fichier intermédiaire **dvf_complet.csv** versionné
Base SQLite **DATAImmo.db** sauvegardée après chaque étape
Script SQL complet conservé (reproductibilité totale)
Fichier de modélisation **.architect** sauvegardé

🔄 **Reproductibilité**
La démarche est entièrement documentée et rejouable
Script SQL en 13 étapes commentées
Procédure Power Query documentée pas à pas
En cas de perte : reconstruction complète possible depuis les sources open data

AA📁 **Organisation des fichiers**
📂 **/sources** — Fichiers bruts originaux
📂 **/traitement** — dvf_complet.csv nettoyé
📂 **/base** — DATAImmo.db · Script SQL
📂 **/modele** — DATAImmo.architect

3 Données initiales

📄 Valeurs-foncières (DVF)

Source DGFiP — Open data

- ~43 colonnes
- → **12 colonnes conservées**
- Filtre : S1 2020 + Valeur > 0

📄 Donnees-communes

Source INSEE

- ~35 000 communes
- 9 colonnes disponibles
- → **5 colonnes conservées**
- Population PTOT = PMUN + PCAP
- Gestion Corse 2A/2B et DOM

📄 Référentiel géographique

Source MEN-ESR

- ~35 000 communes
- 50+ colonnes disponibles
- → **5 colonnes conservées**
- Noms de régions officiels 2016
- 18 régions métropolitaines + DOM

🔧 Transformations appliquées (Power Query)

- Suppression Nom acquéreur (RGPD)
- Filtre dates : 01/01/2020 → 30/06/2020
- Formatage Code_departement sur 2 caractères
- Formatage Code_commune sur 3 caractères
- Renommage de toutes les colonnes
- Suppression valeurs nulles et zéro
- Export CSV UTF-8 : dvf_complet.csv
- Résultat : ~180 000 lignes · 12 colonnes

4 Extrait dictionnaire de données

Extrait des colonnes clés — Tables BIEN et VENTE (issues du fichier Valeurs-foncières)

Table : BIEN

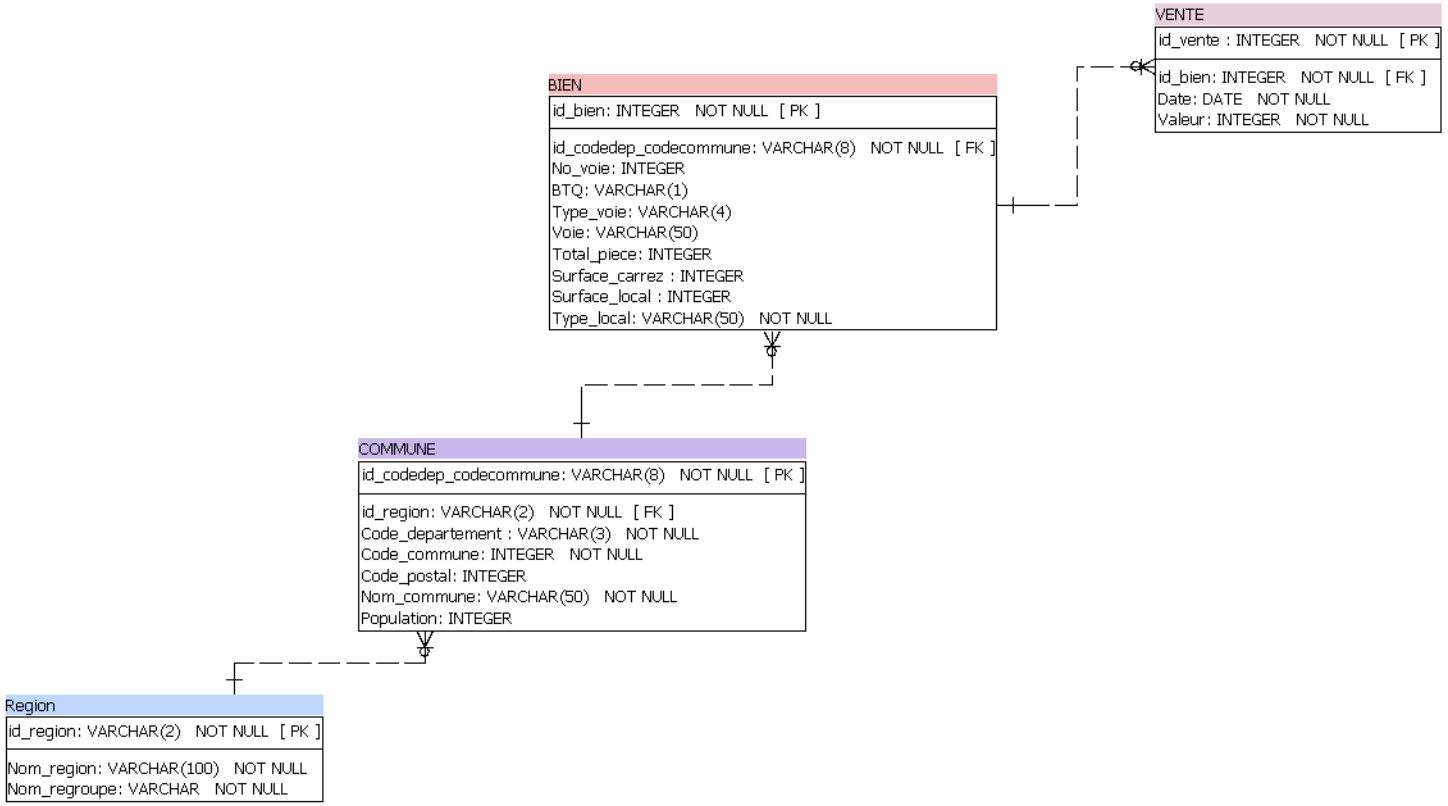
CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	RÈGLE DE GESTION	RÈGLE DE CALCUL
id_bien PK	Identifiant unique auto-généré du bien immobilier	INTEGER	NC	Élémentaire	Ne doit pas être nul · AUTOINCREMENT · Clé primaire	
id_codedep_codecommune FK	Clé étrangère vers la table COMMUNE	VARCHAR	8	Élémentaire	Ne doit pas être nul · Référence COMMUNE(id_codedep_codecommune)	
No_voie	Numéro dans la voie (numéro de rue)	INTEGER	NC	Élémentaire	Peut être nul (absence de numérotation)	
BTQ	Indice de répétition du numéro : Bis / Ter / Quater	VARCHAR	1	Élémentaire	Valeurs admises : B, T, Q ou NULL	
Type_voie	Type de voie (RUE, AV, BD, IMP, CHE…)	VARCHAR	4	Élémentaire	Peut être nul	
Voie	Nom de la voie (libellé de la rue)	VARCHAR	50	Élémentaire	Peut être nul	
Total_piece	Nombre total de pièces principales du bien	INTEGER	NC	Élémentaire	Doit être > 0	

Table : VENTE

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	RÈGLE DE GESTION	RÈGLE DE CALCUL
id_vente PK	Identifiant unique auto-généré de la transaction	INTEGER	NC	Élémentaire	Ne doit pas être nul · AUTOINCREMENT · Clé primaire	
id_bien FK	Clé étrangère vers la table BIEN	INTEGER	NC	Élémentaire	Ne doit pas être nul · Référence BIEN(id_bien)	
Date	Date de la mutation (transaction immobilière)	DATE	NC	Élémentaire	Ne doit pas être nul · Format DD/MM/YYYY dans la source DVF	
Valeur	Valeur foncière de la transaction en euros	INTEGER	NC	Élémentaire	Ne doit pas être nul · Doit être > 0	

📌 **Choix de modélisation — Séparation BIEN / VENTE**

Les caractéristiques du bien (adresse, surface, type) sont séparées de la transaction (date, valeur) pour éliminer toute redondance : un même bien peut être vendu plusieurs fois sansdupliquer ses caractéristiques. Ce choix est conforme à la **3NF**.


VENTE
id_vente : INTEGER NOT NULL [PK]
id_bien: INTEGER NOT NULL [FK]
Date: DATE NOT NULL
Valeur: INTEGER NOT NULL
BIEN
id_bien: INTEGER NOT NULL [PK]
id_codedep_codecommune: VARCHAR(8) NOT NULL [FK]
No_voie: INTEGER
BTQ: VARCHAR(1)
Type_voie: VARCHAR(4)
Voie: VARCHAR(50)
Total_piece: INTEGER
Surface_carrez : INTEGER
Surface_local : INTEGER
Type_local: VARCHAR(50) NOT NULL
COMMUNE
id_codedep_codecommune: VARCHAR(8) NOT NULL [PK]
id_region: VARCHAR(2) NOT NULL [FK]
Code_departement : VARCHAR(3) NOT NULL
Code_commune: INTEGER NOT NULL
Code_postal: INTEGER
Nom_commune: VARCHAR(50) NOT NULL
Population: INTEGER
Region
id_region: VARCHAR(2) NOT NULL [PK]
Nom_region: VARCHAR(100) NOT NULL
Nom_regroupe: VARCHAR NOT NULL

Le schéma relationnel normalisé (3NF)

AA

📐 Pourquoi normaliser ? Le problème de départ

❌ Sans normalisation : 1 seule table géante

- Chaque vente répète le nom de la région, de la commune, le code département…
- "Île-de-France" stocké **120 000 fois** au lieu d'une seule fois
- Si on renomme une région → il faut modifier 120 000 lignes
- Si on se trompe dans 1 ligne → incohérence dans la base
- Taille : ~1 Go de données redondantes

✅ Avec normalisation : 4 tables liées

- "Île-de-France" stocké **1 seule fois** dans la table REGION
- Modification du nom = 1 seule ligne à changer
- Zéro risque d'incohérence entre les lignes
- Taille : ~105 Mo — gain de 90%

AA

AA

5 Le schéma rela

AA

Forme
1NF
2NF
AA**3NF**

La règle
Chaque cellule du tableau contient **une seule valeur atomique** (indivisible). Pas de listes, pas de groupes répétitifs. Chaque table a une clé primaire.

Comment on la respecte
•Surface_carrez = un nombre → ✅ atomique
•Type_local = "Appartement" → ✅ atomique
•On ne stocke pas "Appartement, Maison" dans une cellule
•Chaque table a sa PK unique (id_bien, id_vente...)

Ce qui violerait la 1NF
•Stocker plusieurs lots dans une colonne : "Lot1;Lot2;Lot3"
•Avoir une colonne "Vente1_date, Vente2_date, Vente3_date"
•Pas de PK → impossible d'identifier une ligne

📐 2NF — Deuxième Forme Normale

La règle
Respecte la 1NF + **chaque attribut non-clé dépend de la totalité de la clé primaire**, pas seulement d'une partie. S'applique surtout aux clés composées.

Comment on la respecte
•Dans VENTE : Date et Valeur dépendent de *id_vente* → ✅
•Dans BIEN : Surface dépend de *id_bien* → ✅
•Dans COMMUNE : Nom_commune dépend de *id_codedep_codecommune* → ✅
•Nos PK sont des identifiants simples (sauf COMMUNE) donc pas de dépendance partielle possible

Ce qui violerait la 2NF
•Si on avait une PK composée (id_vente + id_bien) et que Surface ne dépendait que de id_bien : dépendance partielle → violation

📐 3NF — Troisième Forme Normale (notre objectif)

La règle
Respecte la 2NF + **aucun attribut non-clé ne dépend d'un autre attribut non-clé**. En clair : pas de dépendance transitive.
Une dépendance transitive = A → B → C (C dépend de B qui dépend de A, au lieu de dépendre directement de A)

❌ AVANT : Table unique avec dépendance transitive
`VENTE(id_vente, Date, Valeur, Type_local, Surface, Nom_commune, Code_dep, Nom_region)`
•`id_vente` → `Code_dep` → `Nom_region`
•Nom_region ne dépend PAS directement de id_vente
•Il dépend de Code_dep, qui lui dépend de id_vente
•C'est une dépendance transitive → **violation 3NF**

✅ APRÈS : 4 tables séparées — 3NF respectée
•`Nom_region` dépend uniquement de `id_region` (table REGION) ✅
•`Nom_commune` dépend uniquement de `id_codedep_codecommune` (table COMMUNE) ✅

5 Le schéma relationnel normalisé (3NF)

AA

Relation	Côté 1 (parent)	Côté N (enfant)
R1	REGION	COMMUNE
R2	COMMUNE	BIEN
R3	BIEN	VENTE

🏘️ **COMMUNE**
PK id_codedep_codecommune *VARCHAR(8)* Code dept + code commune INSEE
Nom_commune · Code_postal · Population — Données propres à la commune
FK id_region *VARCHAR(2)* → pointe vers REGION.id_region
R2 — 1 commune contient N biens
Un bien est situé dans UNE seule commune

🏠 **BIEN**
PK id_bien *INTEGER AUTO* Généré automatiquement
No_voie · BTQ · Type_voie · Voie — Adresse du bien
Total_piece · Surface_carrez · Surface_local · Type_local — Caractéristiques du bien
FK id_codedep_codecommune *VARCHAR(8)* → pointe vers COMMUNE.id_codedep_codecommune
R3 — 1 bien peut avoir N ventes
Une vente concerne UN seul bien (reventes possibles)

💰 **VENTE**
PK id_vente *INTEGER AUTO* Généré automatiquement
Date · Valeur — Informations de la transaction
FK id_bien *INTEGER* → pointe vers BIEN.id_bien

🔗 **Pourquoi séparer BIEN et VENTE ? (choix crucial)**

❌ **Sans séparation : tout dans une table**
- Un bien vendu 3 fois → ses caractéristiques (surface, adresse, type) répétées 3 fois
- Si la surface est corrigée → 3 lignes à modifier
- Risque d'incohérence : 3 surfaces différentes pour le même bien
- Impossible d'analyser l'évolution du prix d'un bien dans le temps

✅ **Avec séparation BIEN / VENTE**
- Les caractéristiques du bien sont stockées **une seule fois**
- Chaque transaction est une ligne dans VENTE avec un FK vers BIEN
- Correction de surface = 1 seule ligne modifiée
- On peut analyser l'évolution du prix d'un même bien dans le temps

🔗 **Les cardinalités expliquées simplement**

AA

seule commune.

Un appartement peut être vendu en 2018 puis revendu en 2020. Mais chaque vente concerne **un seul** bien.

6 La base de données — Tables créées et données chargées

✅ **Méthode de chargement**

Import CSV → table temporaire DVF_Temp

INSERT DISTINCT → table BIEN (déduplication)

INSERT + JOIN multi-critères → table VENTE

DROP DVF_Temp après alimentation

🔍 **Contrôles d'intégrité**

0 vente orpheline (sans bien associé)

0 bien orphelin (commune introuvable)

FK activées (PRAGMA foreign_keys = ON)

6 index créés pour optimiser les requêtes

7 Requêtes de démonstration — Bon chargement des données

AA

AA

AA

AA

AA

AA

6 Contexte du projet

AA

AA

AA

AA

AA

AA

1. Contexte du projet

🎯 Objectif

Concevoir une base de données relationnelle normalisée (3NF)
Alimenter la base à partir des chiers DVF open data DGFiP
Permettre des analyses du marché immobilier par région, département et commune
Répondre à 12 requêtes d'analyse métier

🛠️ Outils utilisés

Power Query Nettoyage et transformation des données
SQLite Moteur de base de données
DB Browser Interface de gestion SQLite
SQL Power Architect Modélisation du schéma

📂 Sources de données

Valeurs-foncières (DVF) — DGFiP · Transactions immobilières S1 2020
Donnees-communes — INSEE · Population légale des communes
fr-esr-referentiel-geographique — MEN · Référentiel régions/communes

📐 Architecture

Schéma normalisé **3NF** : élimination de toutes les dépendances transitives
4 tables : REGION · COMMUNE · BIEN · VENTE
3 clés étrangères · Intégrité référentielle garantie
Gain volumétrique : **~90%** vs table unique

2 Stratégie de sauvegarde et conformité RGPD

🔒 **Donnée sensible identifiée et supprimée :** "Nom de l'acquéreur" — suppression dès l'étape Power Query, avant tout traitement

⚖️ Conformité RGPD

- ❌ Nom de l'acquéreur supprimé (donnée personnelle directement identifiante)
- ✅ Suppression réalisée dès Power Query (étape 2 du nettoyage)
- ✅ Aucune donnée nominative conservée dans la base finale
- ✅ Les transactions restent analysables sans identifier les personnes

💾 Stratégie de sauvegarde

- ✅ Fichiers sources conservés en version originale (non modifiés)
- ✅ Fichier intermédiaire **dvf_complet.csv** versionné
- ✅ Base SQLite **DATAImmo.db** sauvegardée après chaque étape
- ✅ Script SQL complet conservé (reproductibilité totale)
- ✅ Fichier de modélisation **.architect** sauvegardé

🔄 Reproductibilité

La démarche est entièrement documentée et rejouable Script SQL en 13 étapes

Procédure Power Query documentée pas à pas

En cas de perte : reconstruction complète possible depuis les sources ope

📁 Organisation des fichiers

- 📂 **/sources** — Fichiers bruts originaux
- 📂 **/traitement** — dvf_complet.csv nettoyé
- 📂 **/base** — DATAImmo.db · Script SQL
- 📂 **/modele** — DATAImmo.architect

3. Les données initiales

Valeurs-foncières (DVF)
Source DGFiP — Open data
~500 000 lignes brutes
~43 colonnes
→ **12 colonnes conservées**
Filtre : S1 2020 + Valeur > 0
~180 000 lignes après f i ltre

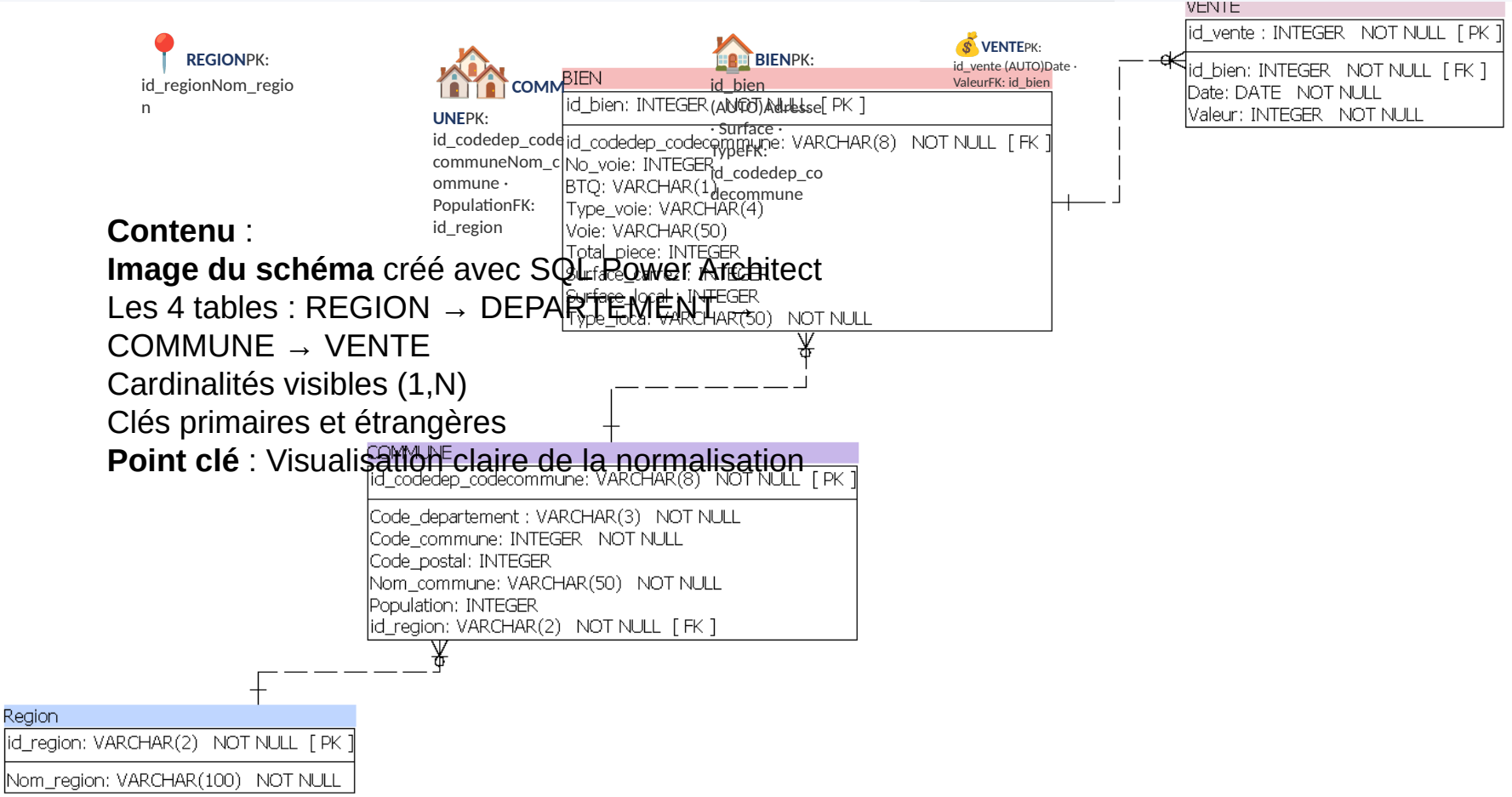
Donnees-communes
Source INSEE
~35 000 communes
9 colonnes disponibles
→ **5 colonnes conservées**
Population PTOT = PMUN + PCAP
Gestion Corse 2A/2B et DOM

Référentiel géographique
Source MEN-ESR
~35 000 communes
50+ colonnes disponibles
→ **5 colonnes conservées**
Noms de régions of f i ciels 2016
18 régions métropolitaines + DOM

Transformations appliquées (Power Query)
Suppression **Nom acquéreur** (RGPD)
Filtre dates : 01/01/2020 → 30/06/2020
Suppression valeurs nulles et zéro

Formatage Code_departement sur 2 caractères
Formatage Code_commune sur 3 caractères
Renommage de toutes les colonnes

Export CSV UTF-8 : dvf_complet.csv
Résultat : ~180 000 lignes · 12 colonnes



Contenu :
Image du schéma créé avec SQL Power Architect
Les 4 tables : REGION → DEPARTEMENT →
COMMUNE → VENTE
Cardinalités visibles (1,N)
Clés primaires et étrangères
Point clé : Visualisation claire de la normalisation

DB Browser for SQLite - C:\

Fichier Édition Vue Outils

Ouvrir un projet Enre

Structure de la base de données

SQL 1*

```
1  SELECT COUNT(*)
2  FROM Vente v
3  LEFT JOIN Bien b
4  WHERE b.id_bien
5
6
7
```

	ventes_orphelines
1	0

DB Browser for SQLite - C:\Users\sabah\OneDrive\Bureau\Projet 5.db

Fichier Édition Vue Outils Aide

Ouvrir un projet Enregistrer le projet

Structure de la base de données | Parcourir les données | Éditer les pragmas | Exécuter le SQL

SQL 1*

```
1  SELECT COUNT(*) AS biens_orphelins
2  FROM Bien b
3  LEFT JOIN Commune c ON b.id_codedep_codecommune = c.id_codedep_codecommune
4  WHERE c.id_codedep_codecommune IS NULL;
5
6
7
```

	biens_orphelins
1	0

Ouvrir un projet | Enregistrer le projet

Structure de la base de données | Parcourir les données | Éditer les pragmas | Exécuter le SQL

Créer une table | Créer un index | Modifier la table | Supprimer la table | Imprimer | Rafraîchir

Nom	Type	Schéma
Tables (6)		
Bien		CREATE TABLE Bien (id_bien INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, id_codedep_codecommune VARCHAR(8) NOT NULL, No_voie INTEGER, BTQ VARCHAR(1), Type_voie VARCHAR(4), Voie VARCHAR(50), Total_piece INTEGER CHECK (Total_
id_bien	INTEGER	"id_bien" INTEGER NOT NULL
id_codedep_codecommune	VARCHAR(8)	"id_codedep_codecommune" VARCHAR(8) NOT NULL
No_voie	INTEGER	"No_voie" INTEGER
BTQ	VARCHAR(1)	"BTQ" VARCHAR(1)
Type_voie	VARCHAR(4)	"Type_voie" VARCHAR(4)
Voie	VARCHAR(50)	"Voie" VARCHAR(50)
Total_piece	INTEGER	"Total_piece" INTEGER CHECK("Total_piece" >= 0)
Surface_carrez	REAL	"Surface_carrez" REAL CHECK("Surface_carrez" > 0)
Surface_local	INTEGER	"Surface_local" INTEGER CHECK("Surface_local" >= 0)
Type_local	VARCHAR(50)	"Type_local" VARCHAR(50) NOT NULL
Commune		CREATE TABLE Commune (id_codedep_codecommune VARCHAR(8) PRIMARY KEY NOT NULL, Code_departement VARCHAR(3) NOT NULL, Code_commune INTEGER NOT NULL, Code_postal INTEGER, Nom_commune VARCHAR(50) NOT NULL, Population I
id_codedep_codecommune	VARCHAR(8)	"id_codedep_codecommune" VARCHAR(8) NOT NULL
Code_departement	VARCHAR(3)	"Code_departement" VARCHAR(3) NOT NULL
Code_commune	INTEGER	"Code_commune" INTEGER NOT NULL
Code_postal	INTEGER	"Code_postal" INTEGER
Nom_commune	VARCHAR(50)	"Nom_commune" VARCHAR(50) NOT NULL
Population	INTEGER	"Population" INTEGER CHECK("Population" >= 0)
id_region	VARCHAR(2)	"id_region" VARCHAR(2) NOT NULL
DVF_Temp		CREATE TABLE DVF_Temp (Code_departement TEXT, Code_commune TEXT, No_voie INTEGER, BTQ TEXT, Type_voie TEXT, Voie TEXT, Total_piece INTEGER, Surface_carrez REAL, Surface_local INTEGER, Type_local TEXT, Date TEXT, Valeur INTEGER)
Code_departement	TEXT	"Code_departement" TEXT
Code_commune	TEXT	"Code_commune" TEXT
No_voie	INTEGER	"No_voie" INTEGER
BTQ	TEXT	"BTQ" TEXT
Type_voie	TEXT	"Type_voie" TEXT
Voie	TEXT	"Voie" TEXT
Total_piece	INTEGER	"Total_piece" INTEGER
Surface_carrez	REAL	"Surface_carrez" REAL
Surface_local	INTEGER	"Surface_local" INTEGER
Type_local	TEXT	"Type_local" TEXT
Date	TEXT	"Date" TEXT
Valeur	INTEGER	"Valeur" INTEGER
Region		CREATE TABLE Region (id_region VARCHAR(2) PRIMARY KEY NOT NULL, Nom_region VARCHAR(100) NOT NULL)
id_region	VARCHAR(2)	"id_region" VARCHAR(2) NOT NULL
Nom_region	VARCHAR(100)	"Nom_region" VARCHAR(100) NOT NULL
Vente		CREATE TABLE Vente (id_vente INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, id_bien INTEGER NOT NULL, Date DATE NOT NULL, Valeur INTEGER NOT NULL CHECK (Valeur > 0), FOREIGN KEY (id_bien) REFERENCES Bien(id_bien))
id_vente	INTEGER	"id_vente" INTEGER NOT NULL
id_bien	INTEGER	"id_bien" INTEGER NOT NULL
Date	DATE	"Date" DATE NOT NULL
Valeur	INTEGER	"Valeur" INTEGER NOT NULL CHECK("Valeur" > 0)

DB Browser for SQLite - C:\Users\sabah\OneDrive\Bureau\Proje

Fichier Édition Vue Outils Aide

Ouvrir un projet Enregistrer le projet

Structure de la base de données Parcourir les données Éditer le

SQL 1*

```
1  SELECT COUNT(*) AS nb_lignes_importees FROM
2
3
```

	nb_lignes_importees
1	19

DB Browser for SQLite - C:\Users\sabah\OneDrive\Bureau\Projet 5.db

Fichier Édition Vue Outils Aide

Ouvrir un projet Enregistrer le projet

Structure de la base de données Parcourir les données Éditer les pragmas Exécuter le SQL

SQL 1*

```
1  SELECT COUNT(*) AS nb_lignes_importees FROM Commune;
2
3
```

	nb_lignes_importees
1	34915

Commune

Accessibility: Unavailable

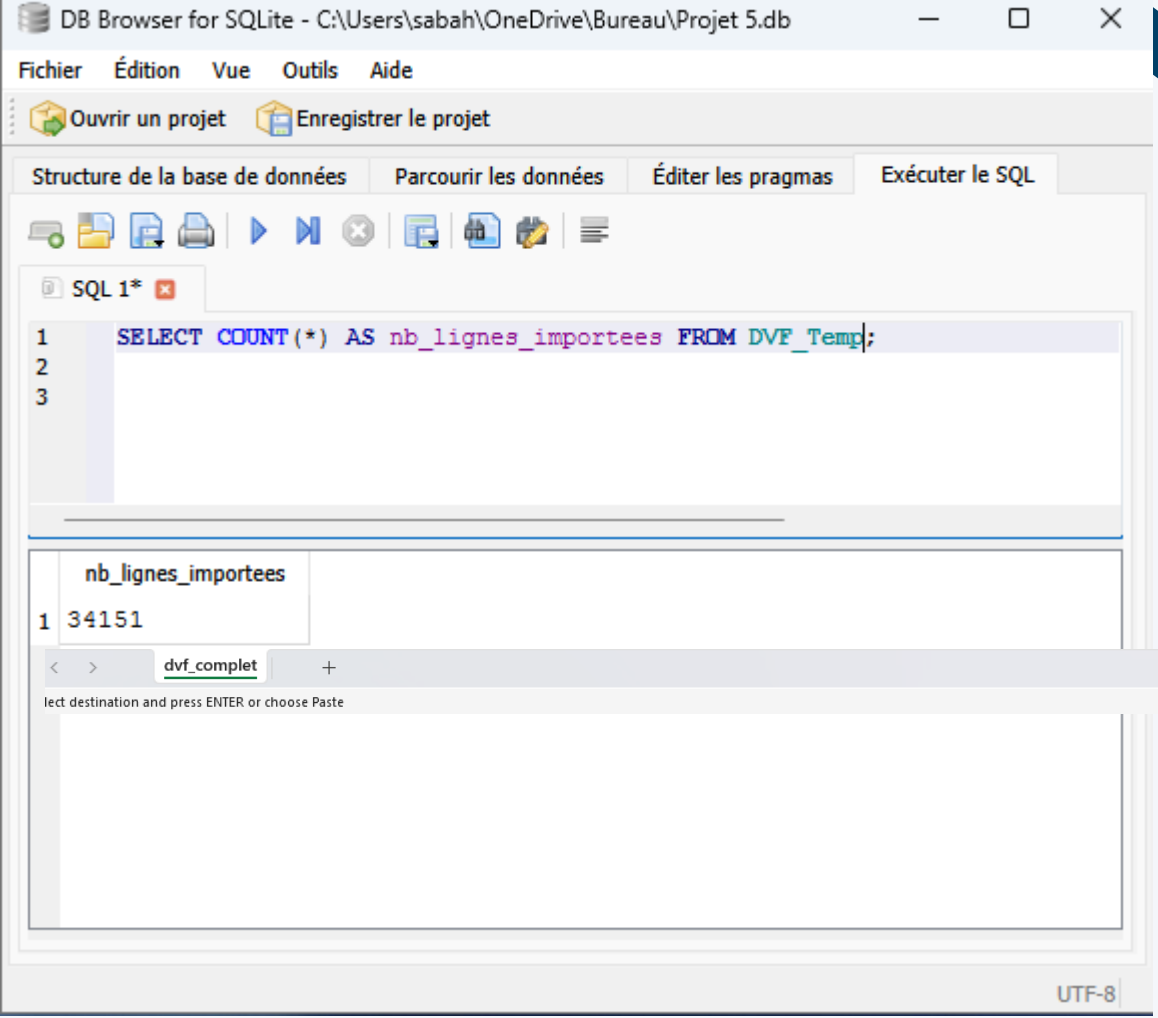
Average: 50664,92219 Count: 34916

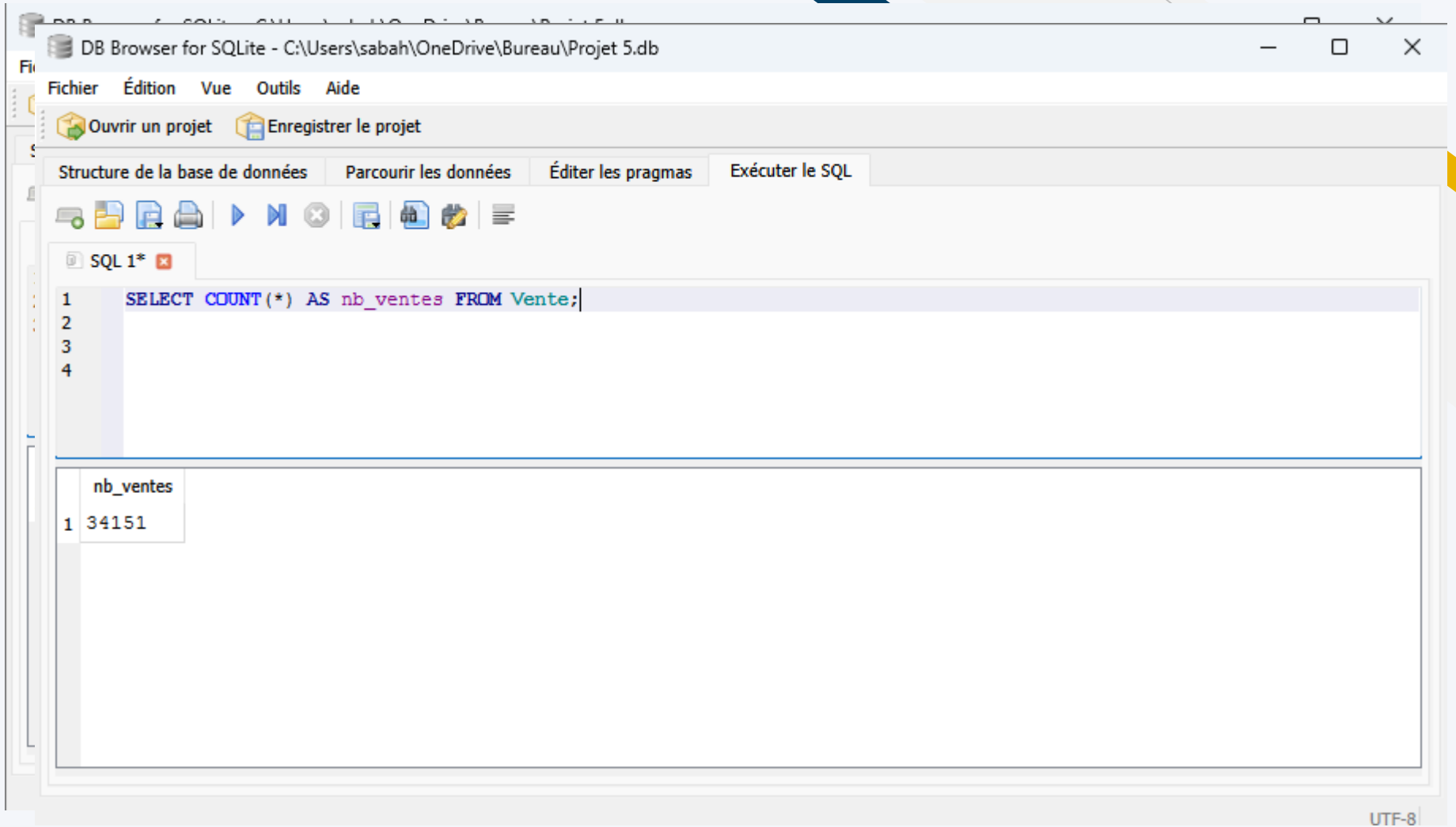
Region

Ready Accessibility: Unavailable

Average: 53,30769231 Count: 20

UTF-8

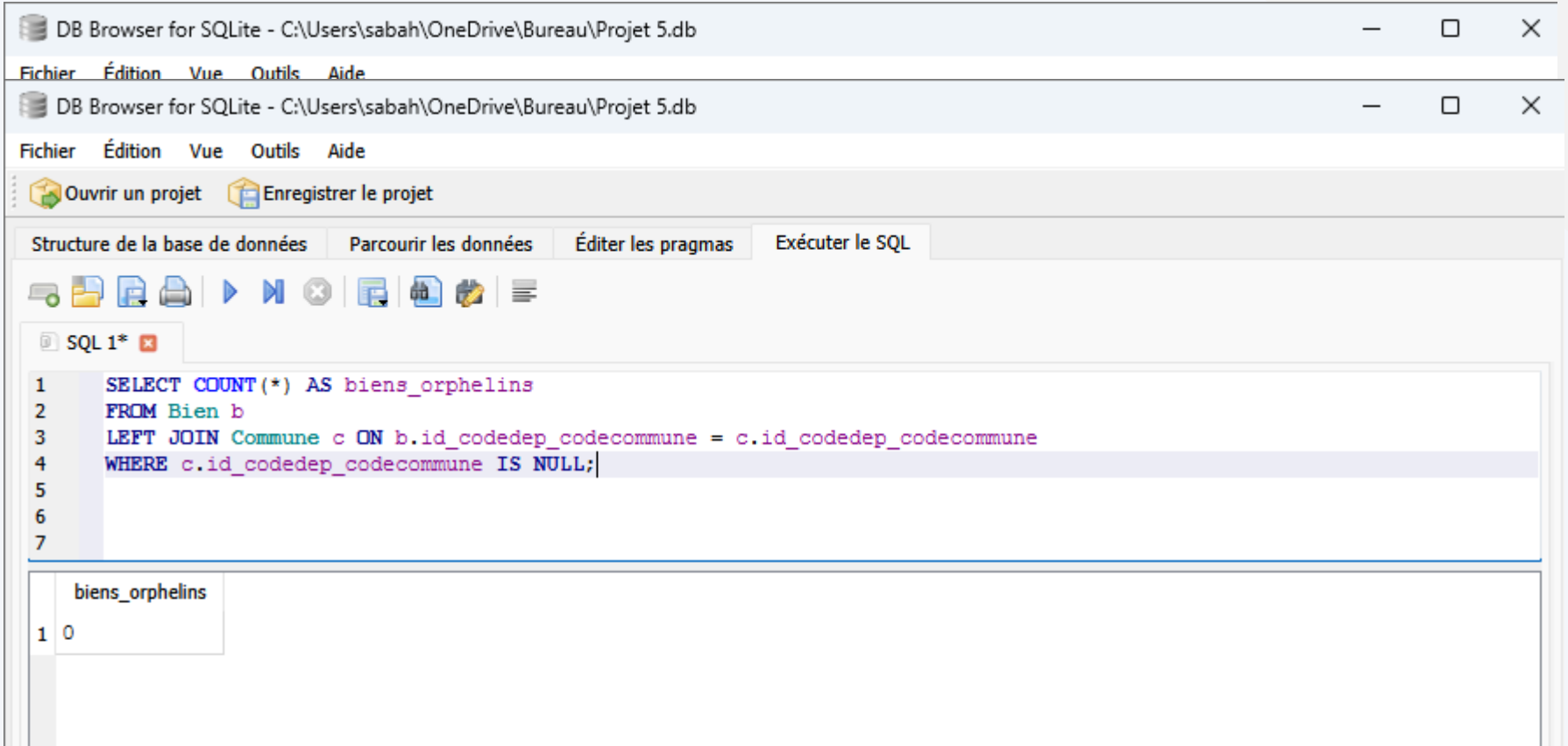

DB Browser for SQLite - C:\Users\sabah\OneDrive\Bureau\Projet 5.db
Fichier
Édition
Vue
Outils
Aide
Ouvrir un projet
Enregistrer le projet
Structure de la base de données
Parcourir les données
Éditer les pragmas
Exécuter le SQL
SQL 1*
1 SELECT COUNT(*) AS nb_lignes_importees FROM DVF_Temp;
2
3
nb_lignes_importees
1 34151
dvf_complet
lect destination and press ENTER or choose Paste
Average: 65,69141852
Count: 34152
UTF-8


DB Browser for SQLite - C:\Users\sabah\OneDrive\Bureau\Projet 5.db
Fichier
Édition
Vue
Outils
Aide
Ouvrir un projet
Enregistrer le projet
Structure de la base de données
Parcourir les données
Éditer les pragmas
Exécuter le SQL
SQL 1*
1 SELECT COUNT(*) AS nb_ventes FROM Vente;
2
3
4
nb_ventes
1 34151
UTF-8

CONTRÔLES D'INTÉGRITÉ RÉFÉRENTIELLE

9a. Ventes orphelines (sans bien associé)

9b. Biens orphelins (commune introuvable)



Récapitulatif des volumes finaux

DB Browser for SQLite - C:\Users\sabah\OneDrive\Bureau\Projet 5.db

Fichier Édition Vue Outils Aide

Ouvrir un projet | Enregistrer le projet

Structure de la base de données | Parcourir les données | Éditer les pragmas | Exécuter le SQL

SQL 1*

```sql
SELECT 'Region'  AS table_name, COUNT(*) AS nb_lignes FROM Region
UNION ALL
SELECT 'Commune',               COUNT(*)              FROM Commune
UNION ALL
SELECT 'Bien',                  COUNT(*)              FROM Bien
UNION ALL
SELECT 'Vente',                 COUNT(*)              FROM Vente;
```

	table_name	nb_lignes
1	Region	19
2	Commune	34915
3	Bien	34135
4	Vente	34151

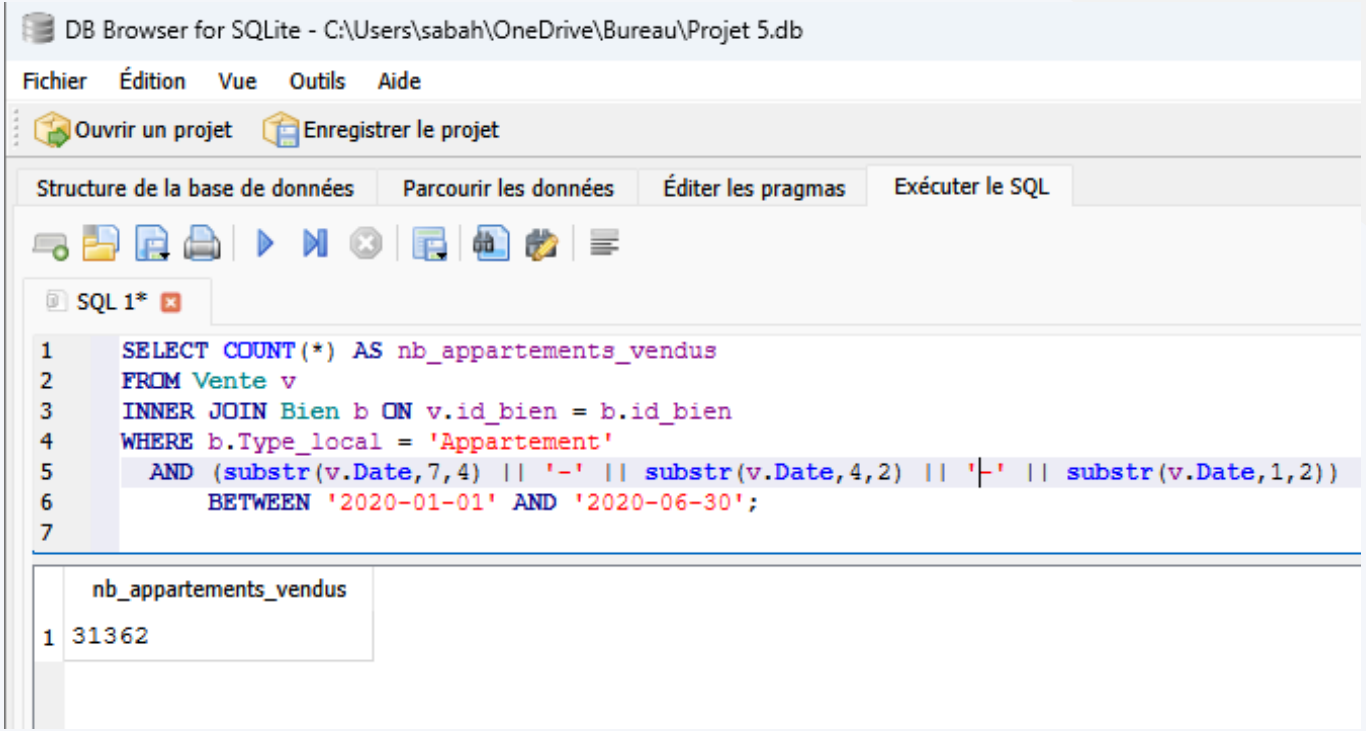


Requêtes SQL et résultats

Sous-titre

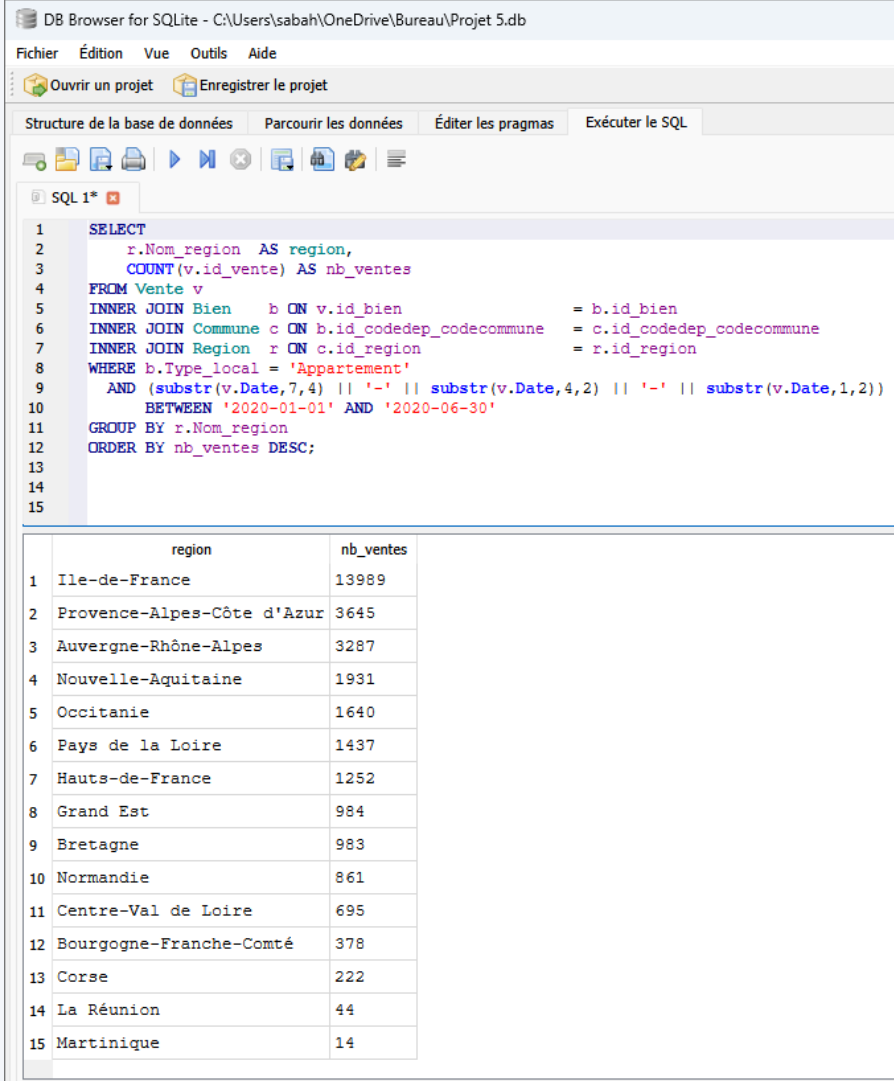
REQUÊTE 1

Nombre total d'appartements vendus au 1er semestre 2020



REQUÊTE 2

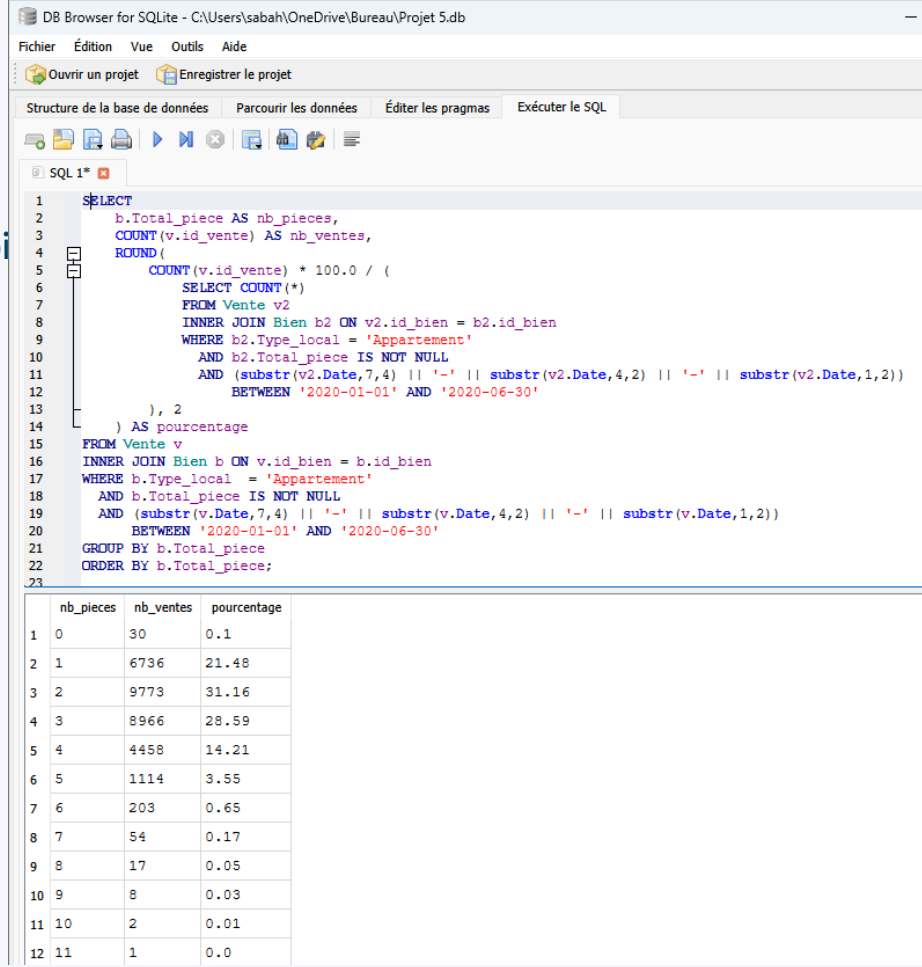
Nombre de ventes d'appartements par région (S1 2020)



	region	nb_ventes
1	Ile-de-France	13989
2	Provence-Alpes-Côte d'Azur	3645
3	Auvergne-Rhône-Alpes	3287
4	Nouvelle-Aquitaine	1931
5	Occitanie	1640
6	Pays de la Loire	1437
7	Hauts-de-France	1252
8	Grand Est	984
9	Bretagne	983
10	Normandie	861
11	Centre-Val de Loire	695
12	Bourgogne-Franche-Comté	378
13	Corse	222
14	La Réunion	44
15	Martinique	14

Requête 3

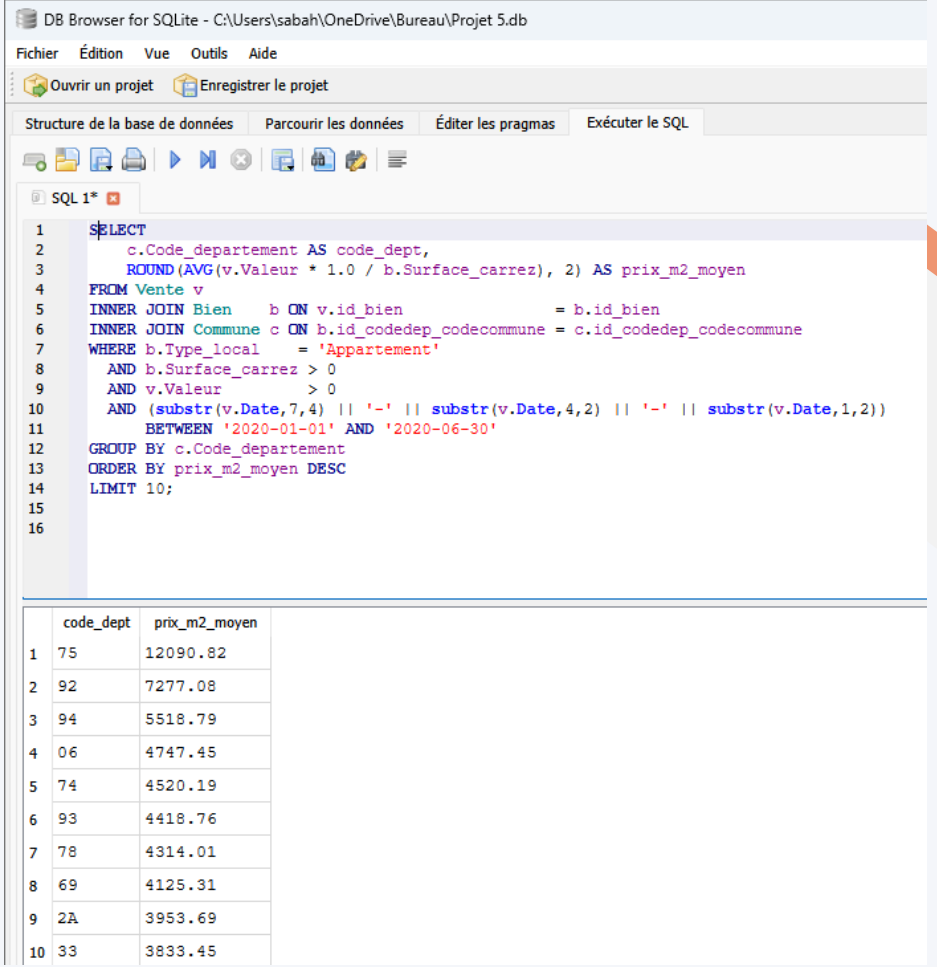
Proportion des ventes d'appartements par nombre de pi



	nb_pieces	nb_ventes	pourcentage
1	0	30	0.1
2	1	6736	21.48
3	2	9773	31.16
4	3	8966	28.59
5	4	4458	14.21
6	5	1114	3.55
7	6	203	0.65
8	7	54	0.17
9	8	17	0.05
10	9	8	0.03
11	10	2	0.01
12	11	1	0.0

Requête 4

Top 10 départements avec le prix au m² le plus élevé

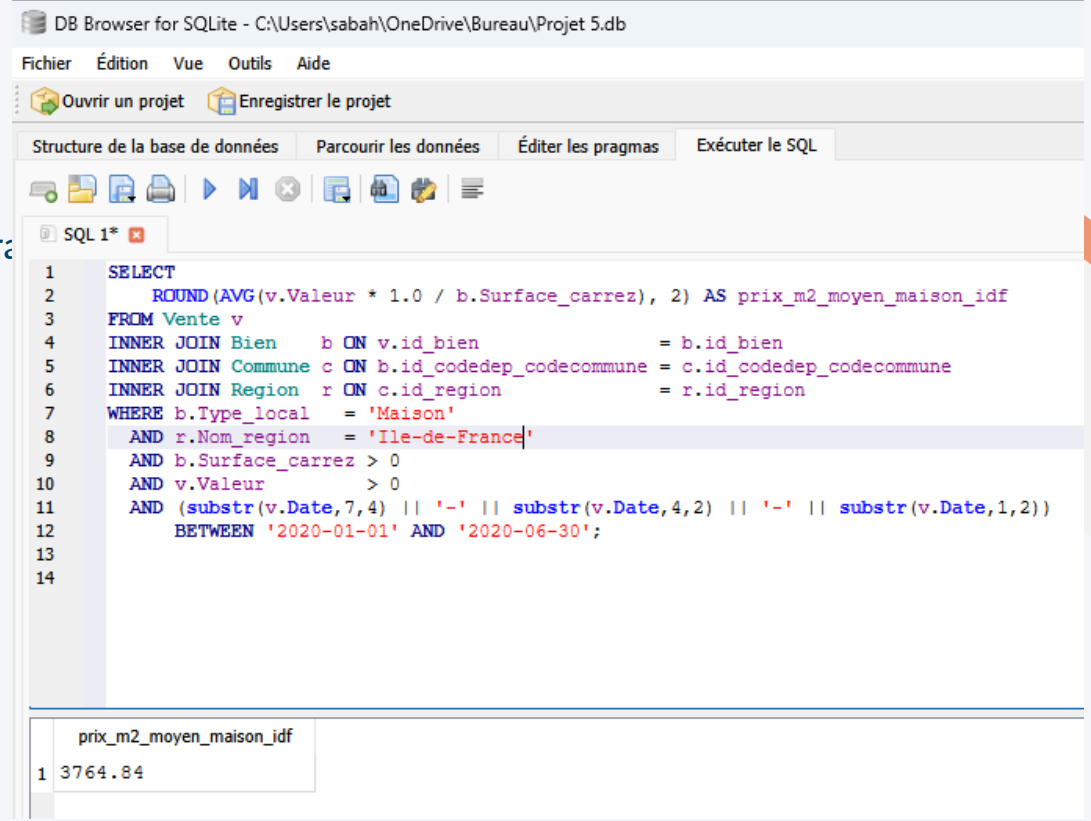


	code_dept	prix_m2_moyen
1	75	12090.82
2	92	7277.08
3	94	5518.79
4	06	4747.45
5	74	4520.19
6	93	4418.76
7	78	4314.01
8	69	4125.31
9	2A	3953.69
10	33	3833.45

Requête 5

Prix moyen du m² d'une maison en Île-de-Fra

- Lorem ipsum



Requête 6

Top 10 appartements les plus chers (avec région et su

DB Browser for SQLite - C:\Users\sabah\OneDrive\Bureau\Projet 5.db

Fichier Édition Vue Outils Aide

Ouvrir un projet | Enregistrer le projet

Structure de la base de données | Parcourir les données | Éditer les pragmas | Exécuter le SQL

SQL 1*

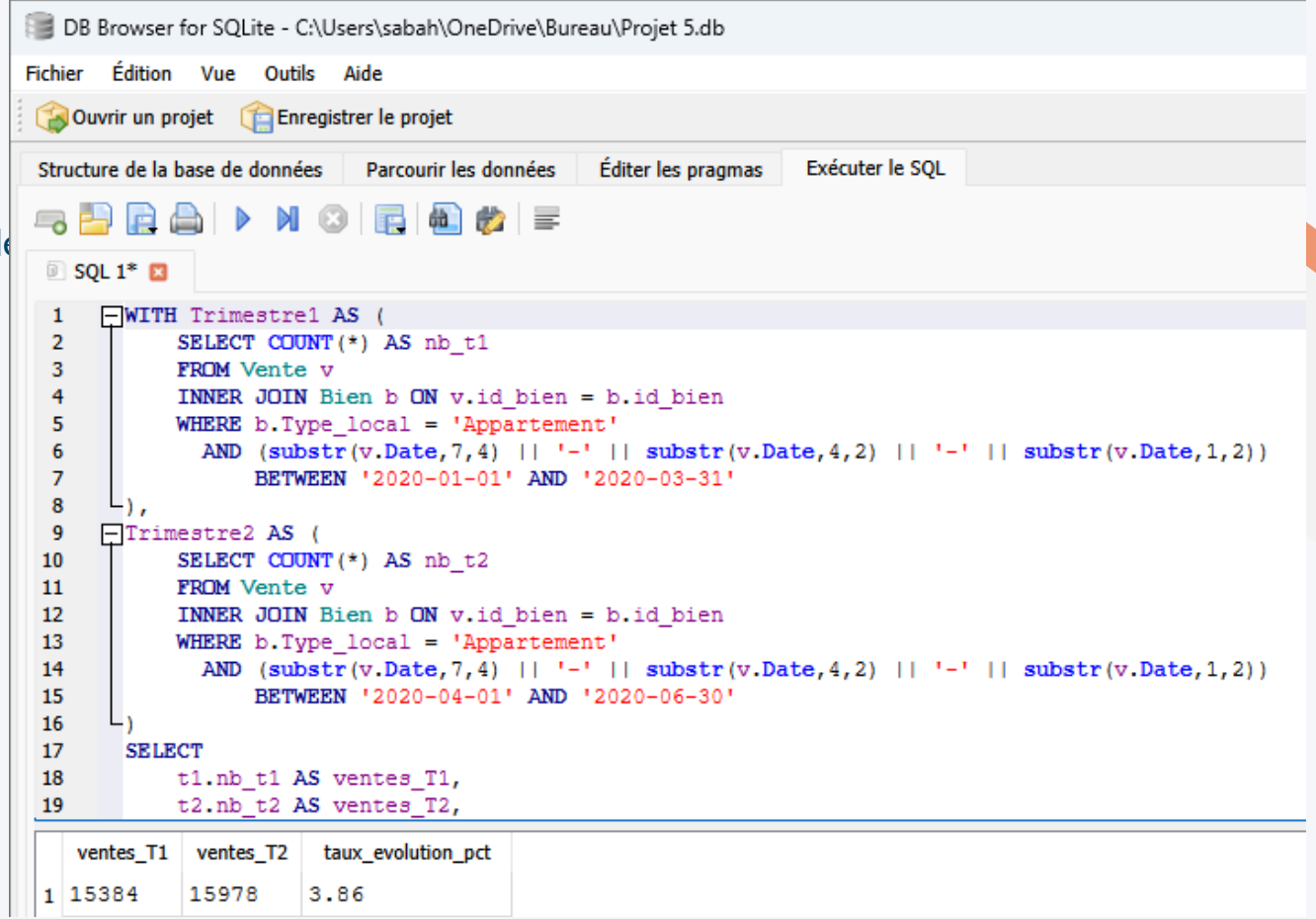
```sql
SELECT
    v.Valeur AS prix_vente,
    b.Surface_carrez AS surface_m2,
    ROUND(v.Valeur * 1.0 / b.Surface_carrez, 2) AS prix_m2,
    b.Total_piece AS nb_pieces,
    c.Nom_commune AS commune,
    c.Code_departement AS departement,
    r.Nom_region AS region
FROM Vente v
INNER JOIN Bien    b ON v.id_bien = b.id_bien
INNER JOIN Commune c ON b.id_codedep_codecommune = c.id_codedep_codecommune
INNER JOIN Region  r ON c.id_region = r.id_region
WHERE b.Type_local    = 'Appartement'
  AND b.Surface_carrez > 0
  AND (substr(v.Date,7,4) || '-' || substr(v.Date,4,2) || '-' || substr(v.Date,1,2))
      BETWEEN '2020-01-01' AND '2020-06-30'
ORDER BY v.Valeur DESC
LIMIT 10;
```

	prix_vente	surface_m2	prix_m2	nb_pieces	commune	departement	region
1	9000000	9,1	1000000.0	1	Paris 16e Arrondissement	75	Ile-de-France
2	8600000	64.0	134375.0	3	Corbeil-Essonnes	91	Ile-de-France
3	8577713	20,55	428885.65	7	Paris 7e Arrondissement	75	Ile-de-France
4	7620000	42,77	181428.57	2	Paris 17e Arrondissement	75	Ile-de-France
5	7600000	253,3	30039.53	5	Paris 6e Arrondissement	75	Ile-de-France
6	7535000	139,9	54208.63	4	Paris 1er Arrondissement	75	Ile-de-France
7	7420000	360,95	20611.11	8	Paris 16e Arrondissement	75	Ile-de-France
8	7200000	595.0	12100.84	8	Paris 16e Arrondissement	75	Ile-de-France
9	7050000	122,56	57786.89	7	Paris 1er Arrondissement	75	Ile-de-France
10	6600000	79,38	83544.3	3	Paris 1er Arrondissement	75	Ile-de-France

Requête 7

Taux d'évolution du nombre de

- Lorem ipsum



Requête 8

Classement des régions par prix/m² (appartements > 4 piè

DB Browser for SQLite - C:\Users\sabah\OneDrive\Bureau\Projet 5.db

Fichier Édition Vue Outils Aide

Ouvrir un projet Enregistrer le projet

Structure de la base de données | Parcourir les données | Éditer les pragmas | Exécuter le SQL

SQL 1*

```sql
SELECT
    r.Nom_region AS region,
    COUNT(v.id_vente) AS nb_ventes,
    ROUND(AVG(v.Valeur * 1.0 / b.Surface_carrez), 2) AS prix_m2_moyen,
    ROUND(AVG(b.Surface_carrez), 2) AS surface_moyenne
FROM Vente v
INNER JOIN Bien    b ON v.id_bien = b.id_bien
INNER JOIN Commune c ON b.id_codedep_codecommune = c.id_codedep_codecommune
INNER JOIN Region  r ON c.id_region = r.id_region
WHERE b.Type_local    = 'Appartement'
  AND b.Total_piece   > 4
  AND b.Surface_carrez > 0
  AND v.Valeur         > 0
  AND (substr(v.Date,7,4) || '-' || substr(v.Date,4,2) || '-' || substr(v.Date,1,2))
      BETWEEN '2020-01-01' AND '2020-06-30'
GROUP BY r.Nom_region
ORDER BY prix_m2_moyen DESC;
```

	region	nb_ventes	prix_m2_moyen	surface_moyenne
1	Ile-de-France	684	8819.54	120.63
2	La Réunion	2	3659.83	120.5
3	Provence-Alpes-Côte d'Azur	117	3616.71	111.47
4	Corse	5	3117.88	128.0
5	Auvergne-Rhône-Alpes	185	2903.86	120.45
6	Nouvelle-Aquitaine	68	2476.5	110.13
7	Bretagne	37	2427.14	102.95
8	Pays de la Loire	55	2297.31	111.35
9	Hauts-de-France	40	2199.92	120.7
10	Occitanie	47	2107.24	105.21
11	Normandie	26	2026.31	104.0
12	Grand Est	76	1560.91	108.33
13	Centre-Val de Loire	31	1459.98	99.61
14	Bourgogne-Franche-Comté	26	1260.73	103.54

Requête 9

Communes ayant eu au moins 50 ventes au T1 2020

Paris 17e Arrondissement	75	228
Paris 15e Arrondissement	75	215
Paris 18e Arrondissement	75	209
Nice	06	173
Paris 11e Arrondissement	75	169
Paris 16e Arrondissement	75	165
Bordeaux	33	157
Paris 14e Arrondissement	75	146
Paris 20e Arrondissement	75	127
Nantes	44	119
Paris 19e Arrondissement	75	116
Paris 12e Arrondissement	75	110
Paris 10e Arrondissement	75	109
Grenoble	38	106
Paris 9e Arrondissement	75	106
Boulogne-Billancourt	92	99
Paris 13e Arrondissement	75	94
Paris 7e Arrondissement	75	87
Paris 6e Arrondissement	75	86
Asnières-sur-Seine	92	81
Marseille 8e Arrondissement	13	81
Courbevoie	92	80
Paris 3e Arrondissement	75	79
Paris 5e Arrondissement	75	79
Toulouse	31	78
Antibes	06	77
Marseille 4e Arrondissement	13	72
Marseille 1er Arrondissement	13	71
Rueil-Malmaison	92	68
Vincennes	94	68
Lille	59	67
Marseille 9e Arrondissement	13	66
Montreuil	93	65
Angers	49	64
Nîmes	30	63
La Ciotat	13	62
Paris 8e Arrondissement	75	62
Sète	34	62
Paris 2e Arrondissement	75	61
Rennes	35	61
Levallois-Perret	92	59
Paris 4e Arrondissement	75	59
Toulon	83	59
Saint-Maur-des-Fossés	94	56
Ajaccio	2A	54
Versailles	78	54
Puteaux	92	53
Issy-les-Moulineaux	92	50

DB Browser for SQLite - C:\Users\sabah\OneDrive\Bureau\Projet 5.db

Fichier Édition Vue Outils Aide

Ouvrir un projet | Enregistrer le projet

Structure de la base de données | Parcourir les données | Éditer les pragmas | Exécuter le SQL

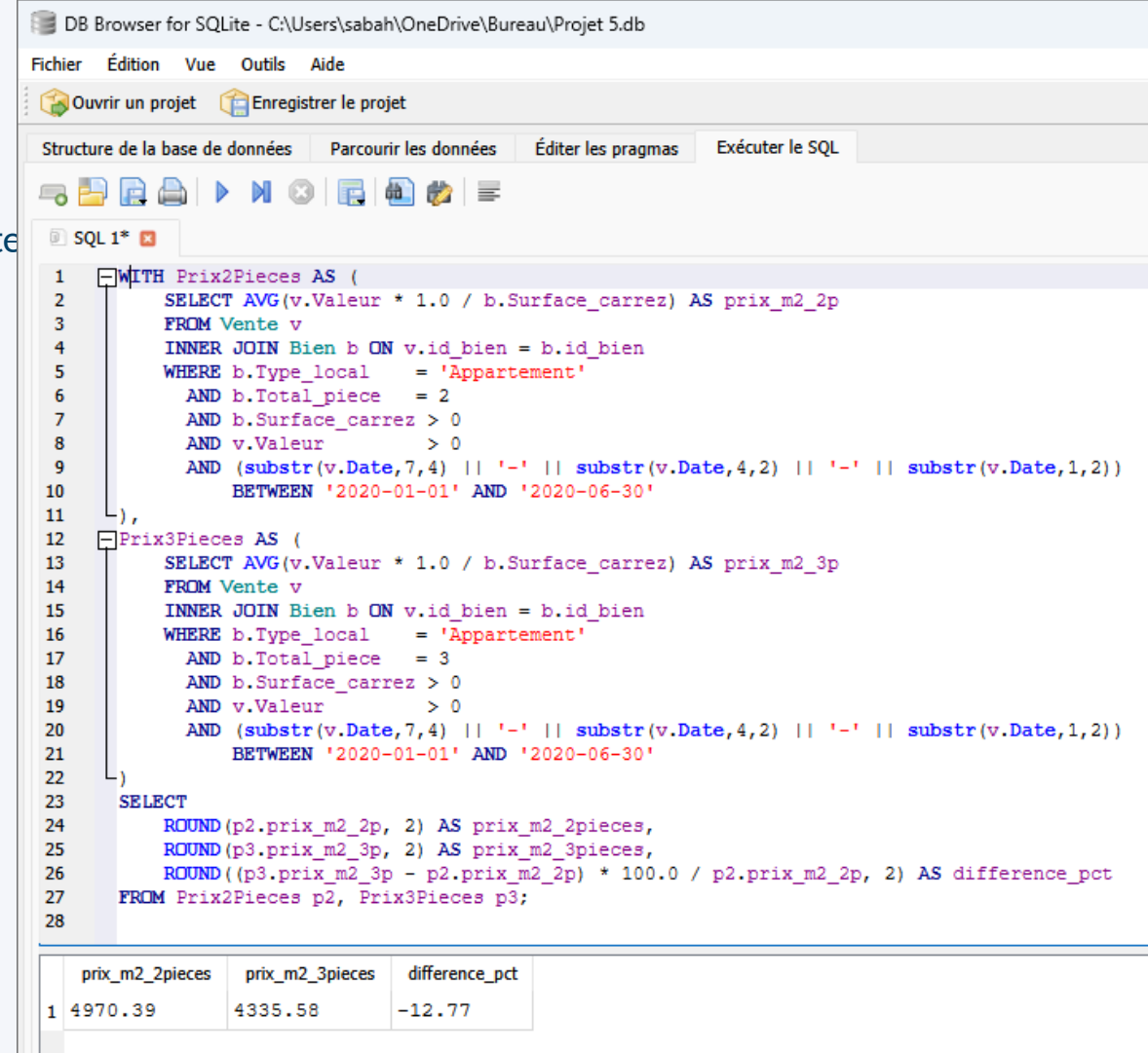
SQL 1*

```sql
SELECT
    c.Nom_commune      AS commune,
    c.Code_departement AS dept,
    COUNT(v.id_vente)  AS nb_ventes
FROM Vente v
INNER JOIN Bien    b ON v.id_bien                = b.id_bien
INNER JOIN Commune c ON b.id_codedep_codecommune = c.id_codedep_codecommune
WHERE (substr(v.Date,7,4) || '-' || substr(v.Date,4,2) || '-' || substr(v.Date,1,2))
      BETWEEN '2020-01-01' AND '2020-03-31'
GROUP BY c.Nom_commune, c.Code_departement
HAVING COUNT(v.id_vente) >= 50
ORDER BY nb_ventes DESC;
```

	commune	dept	nb_ventes
1	Paris 17e Arrondissement	75	228
2	Paris 15e Arrondissement	75	215
3	Paris 18e Arrondissement	75	209
4	Nice	06	173
5	Paris 11e Arrondissement	75	169
6	Paris 16e Arrondissement	75	165
7	Bordeaux	33	157
8	Paris 14e Arrondissement	75	146
9	Paris 20e Arrondissement	75	127
10	Nantes	44	119
11	Paris 19e Arrondissement	75	116
12	Paris 12e Arrondissement	75	110
13	Paris 10e Arrondissement	75	109
14	Grenoble	38	106
15	Paris 9e Arrondissement	75	106
16	Boulogne-Billancourt	92	99
17	Paris 13e Arrondissement	75	94
18	Paris 7e Arrondissement	75	87
19	Paris 6e Arrondissement	75	86
20	Asnières-sur-Seine	92	81

Requête 10

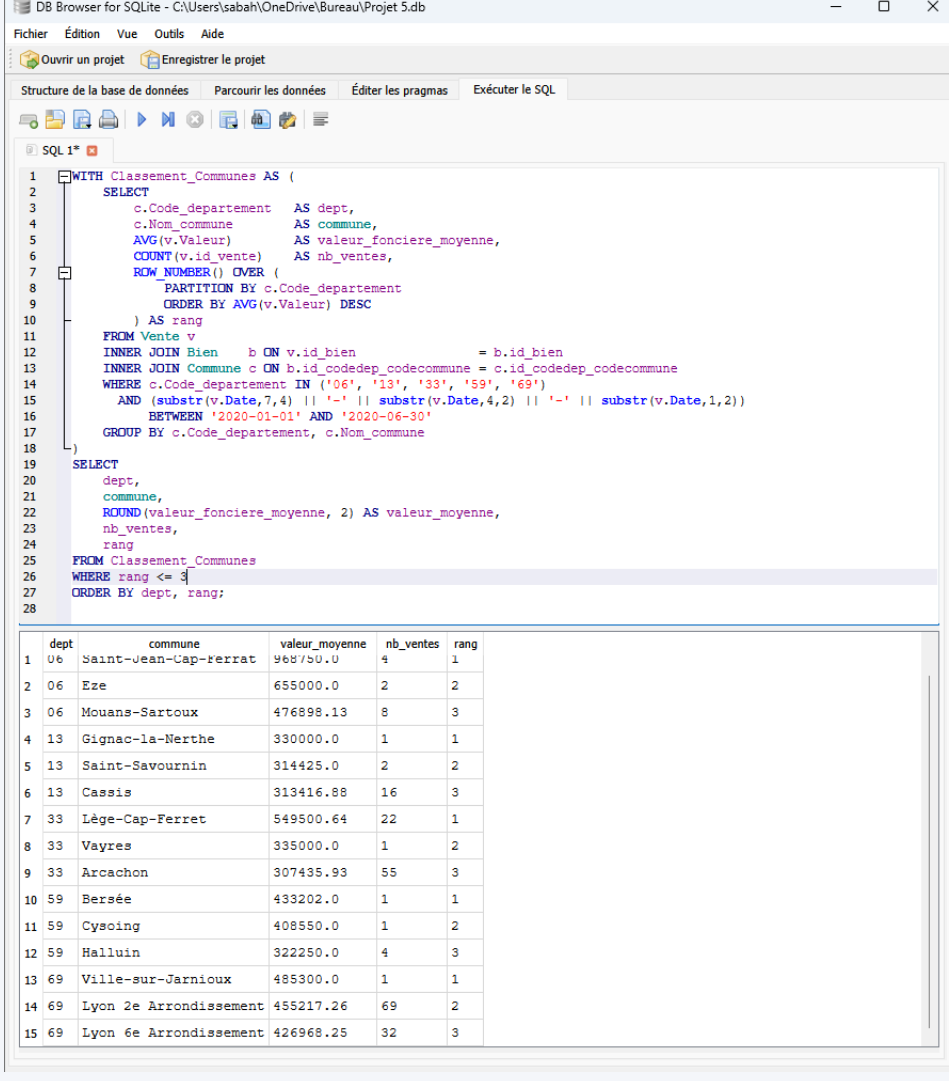
Différence en % du prix/m² entre apparte



Requête 11

Top 3 communes par département (06, 13, 33, 59, 69)

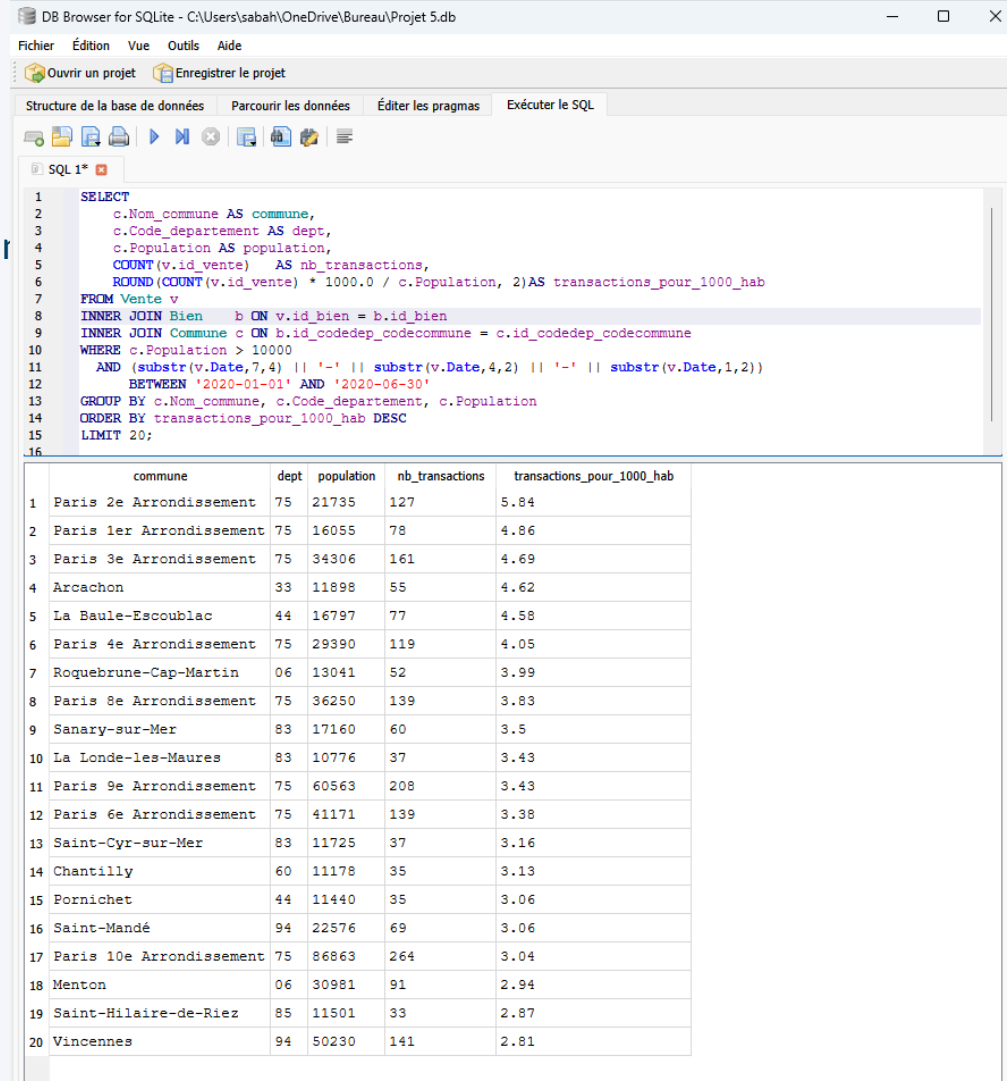
- Lorem ipsum



Requête 12

Top 20 communes : transactions pour 1 000 habitar

- Lorem ipsum



	commune	dept	population	nb_transactions	transactions_pour_1000_hab
1	Paris 2e Arrondissement	75	21735	127	5.84
2	Paris 1er Arrondissement	75	16055	78	4.86
3	Paris 3e Arrondissement	75	34306	161	4.69
4	Arcachon	33	11898	55	4.62
5	La Baule-Escoublac	44	16797	77	4.58
6	Paris 4e Arrondissement	75	29390	119	4.05
7	Roquebrune-Cap-Martin	06	13041	52	3.99
8	Paris 8e Arrondissement	75	36250	139	3.83
9	Sanary-sur-Mer	83	17160	60	3.5
10	La Londe-les-Maures	83	10776	37	3.43
11	Paris 9e Arrondissement	75	60563	208	3.43
12	Paris 6e Arrondissement	75	41171	139	3.38
13	Saint-Cyr-sur-Mer	83	11725	37	3.16
14	Chantilly	60	11178	35	3.13
15	Pornichet	44	11440	35	3.06
16	Saint-Mandé	94	22576	69	3.06
17	Paris 10e Arrondissement	75	86863	264	3.04
18	Menton	06	30981	91	2.94
19	Saint-Hilaire-de-Riez	85	11501	33	2.87
20	Vincennes	94	50230	141	2.81

Résultats clés - Insights métier

Découvertes principales :

Volume : [X] appartements vendus au S1 2020

Géographie : Île-de-France = [X]% des transactions

Prix : Paris en tête à [X] €/m²

Typologie : Les 2 pièces = [X]% du marché

COVID-19 : Baisse de [X]% entre T1 et T2

Dynamisme : [Commune X] en tête du ratio transactions/habitants

Valeur pour Laplace Immo :

Ciblage géographique optimisé

Barèmes d'estimation affinés

Détection des opportunités de marché

Auto-évaluation

Points forts ✅

Normalisation 3NF parfaitement respectée

Conformité RGPD totale

12 requêtes fonctionnelles et optimisées

Documentation exhaustive

Points d'amélioration 🔄

[Optionnel : mentionne 1-2 points si pertinents]

Exemple : "Automatisation de l'import avec scripts Python"

Exemple : "Ajout d'une interface utilisateur graphique"

Compétences acquises 💪

Modélisation relationnelle avancée

Maîtrise SQL (JOIN, agrégations, CTE)

Gestion de données hétérogènes

Conclusion et perspectives

Livrables produits :

✅ Base de données SQLite opérationnelle

✅ Dictionnaire de données complet

✅ Schéma relationnel normalisé

✅ 12 requêtes SQL documentées

✅ Documentation technique complète

Perspectives d'évolution :

📈 Extension à l'année complète 2020

🔄 Automatisation des imports mensuels

📊 Création de tableaux de bord (Power BI / Tableau)

🤖 Intégration d'algorithmes de prédiction de prix



Merci !